

AIPL ユーザーズマニュアル

Actor-based Intelligent Parallel Language

児玉靖

yaskodama@gmail.com

2026 年 8 月 23 日版

Python 実装 (Py-I) 向け。核の仕様は abclcp のユーザーズガイドが正典

Abstract

AIPL (Actor-based Intelligent Parallel Language; 旧称 ABCL/c+) は, 東京工業大学・米澤明憲研究室で設計された並行オブジェクト指向言語 ABCL/1 を, 現代的なシンタックスとマルチランタイム (OCaml ネイティブ・ブラウザ JS・Node.js・C) で再実装した言語である。本マニュアルは言語仕様の概要と, 付属サンプルの解説, 組み込み関数のリファレンスをまとめる。

後方互換のため, ファイル拡張子 `.aipl`, Python ランタイムのモジュール名 (`aipl_main.py` 等), 環境変数 `ABCL_AI_PROVIDER` などは旧称のまま維持している。

対象バージョン: 本リポジトリ (`abclcp-project`) 同梱の OCaml REPL 実装
主要ソース: `src/lexer.mll`, `src/parser.mly`, `src/eval_thread.ml`
サンプル群: `abclcp/*.aipl`, `src/*.aipl`

Contents

1	はじめに	3
2	処理系のビルドと実行	3
2.1	ビルド	3
2.2	サンプル実行	4
2.3	ブラウザ実行	4
3	言語仕様	4
3.1	字句要素	4
3.2	型と値	5
3.3	式と文	5
3.4	クラスとアクター	5
3.5	メッセージ送信	6
3.6	<code>select</code> による選択受信	7
3.7	振る舞いの切り替え — <code>become</code> は無くなった	7
3.8	<code>reply</code> と <code>sender</code>	7
3.9	リモート送信	8
3.10	レコードと組型 (タプル)	8
4	核の七つの規律	8
4.1	待ちには期限が要る	9
4.2	待ちはず「上」へ向かう — 義務レベル	9
4.3	資源は対で取り, 順序を守る	9
4.4	やり取りの順序 — セッション型	9
4.5	<code>sender</code> は Python 版だけの機能である	9

5	組込み関数リファレンス	9
5.1	入出力	10
5.2	制御	10
5.3	数学関数	10
5.4	配列	10
5.5	SDL 描画	10
5.6	Web ゲートウェイ	10
5.7	AI 呼び出し (ai_call)	11
5.8	デバッグ	11
6	サンプルプログラム	11
6.1	Hello — 最小サンプル	11
6.2	Counter — 自己メッセージとフィールド	12
6.3	PingPong — 二者間メッセージ往復	12
6.4	Door — 状態フィールドによる振る舞いの切り替え	13
6.5	Records — レコード型	13
6.6	Tuples — 組型	13
6.7	BoundedBuffer — 生産者・消費者問題	14
6.8	Philosophers — 食事する哲学者	14
6.9	Rotate4Lines — SDL 描画とタイマー	14
6.10	WebCalc — HTTP ゲートウェイと select	15
7	Phase C-E2 型推論 (Python ランタイム)	16
7.1	CLI: --infer と --check	16
7.2	推論される情報	16
7.3	達成された機能 (Phase C → E-2)	17
7.4	where 句の使い方	17
7.5	詳細ドキュメント	17
8	AIPL v2 Distributed Runtime (aipl_dist)	17
8.1	マスター切替	18
8.2	8 機能の一覧 (AIPL_DIST_ENABLE=1 前提)	18
8.3	クイックスタート	18
8.4	PsiLang v3 silent-hang シナリオへの対策一式	18
8.5	サンプル (8 機能 × 3 = 24)	19
8.6	公開 API (aipl_dist モジュール)	19
8.7	進化計算による設計探索の出自	20
9	Next-gen 機能 (CE-10~13 + DR-10~13, 2026-05-18)	20
9.1	8 機能と既定値	21
9.2	CE-10 effect 推論の出力例	21
9.3	CE-11 capability の使い方	21
9.4	DR-11 saga 構文	22
9.5	DR-10 CRDT の使い方	22
9.6	DR-12 multi-region failover	22
9.7	DR-13 auto-scaling actor pool	23
9.8	6 ランタイムでの動作	23
9.9	進化計算による設計の経緯	23
10	付録: トラブルシューティング	24

1 はじめに

このマニュアルの位置づけ

AIPL には実装が三つある (OCaml 版・Python 版・ブラウザの JavaScript 版). ただし三つが同じ言語というわけではない.

層	どこが実装するか	内容
核	三実装すべて	アクターと三種の送信, <code>reply</code> , 型・効果・期限, 失敗の型 <code>result<τ></code> , 返信先の一級性, 資源の順序, 義務レベル, セッション型, メッシュ配備
拡張	Python 版のみ	レコード型と行多相, <code>where</code> による精緻化型, CSP チャンネル, 所有 (<code>pub / move</code>), タプル, <code>match</code> , <code>saga</code> , 分散ランタイム <code>aipl_dist</code>

本書は Python 版のマニュアルである. 核と拡張の両方を扱う. 核だけを厳密に知りたい場合は, [aios/abclcp/docs/aipl_guide_ja.pdf](#) (ユーザーズガイド) を見ること. そちらが仕様の正典で, 三実装で出力の一致まで検証してある.

本書のうち, レコード型・組型・Phase C-E2 型推論・`aipl_dist`・Next-gen 機能の各章は **拡張層**である. OCaml 版と JavaScript 版では動かない.

どちらの層に属するかは機械的に分かる.

```
sh ~/aios/abclcp/tools/classify_corpus.sh
```

OCaml 版が通せば核, Python 版だけが通せば拡張である.

言語の骨格

AIPL は **アクターモデル**に基づく並行プログラミング言語である. プログラムは複数の独立したアクター (クラスのインスタンス) から構成され, アクター同士は **非同期メッセージ**によって通信する.

- 各アクターは固有のメッセージキュー (メールボックス) を持つ
- 状態 (フィールド) はアクター内に閉じ込められ, 外部から直接読み書きできない
- メソッド呼び出しは **メッセージ送信 (send)** として表現される
- 受信側は到着順, または `select` 文で条件付きに処理する

伝統的な ABCL/1 にあった `past/now/future` の三種類の送信モードを, AIPL では `send (past, fire-and-forget)` / `now expr.m()` (同期返答) / `future expr.m() + await` (非同期返答) として完全に提供する.

2 処理系のビルドと実行

2.1 ビルド

トップディレクトリで `make` を実行すると OCaml 実装と JS 実装の両方をビルドする.

```
make          # = make all (OCaml + JS)
make ocaml    # OCaml REPL のみ (_build/default/src/repl_thread.exe)
make js       # ブラウザ用パーサ再生成
```

2.2 サンプル実行

```
make run-hello          # abcl/Hello.aipl
make run-philosophers   # 5 哲学者
make run-rotate4        # SDL 描画デモ
./run_bounded_buffer.sh # 有界バッファ
./run_pingpong_xinu.sh  # PingPong (XINU バックエンド)
```

任意のソースを直接食わせるには:

```
_build/default/src/repl_thread.exe abcl/Hello.aipl
```

2.3 ブラウザ実行

src/browser-abcl/ 以下にブラウザ版がある。make serve-js で <http://localhost:3000/> から各デモ HTML (viz_philosophers.html 等) が開ける。

3 言語仕様

3.1 字句要素

コメント

```
// 行コメント
/* ブロック
   コメント */
```

キーワード

```
class method var float new replyto
send send! call remote reply
if then else while do
select case timeout ->
self sender now future await
```

演算子

種別	演算子
算術	+ - * /
比較	== != < <= > >=
代入	=
フィールド/インデックス	. [n]

+ は数値加算と文字列連結の両方に使われる。片方が文字列なら他方は自動で文字列化される ("count = " + 5)。

リテラル

- 整数: 0, 42
- 浮動小数: 3.14, 100. (末尾ドット可)
- 文字列: "hello\n" (エスケープは \\, \", \n, \t, \r)
- 識別子: [A-Za-z_] [A-Za-z0-9_]*

3.2 型と値

AIPL は **Hindley-Milner 型推論** を中核に持ちつつ、動的値表現を併用する。代表的な型を以下に示す。

型名	例	備考
int	42	64bit 整数
float	3.14, 100.	倍精度
string	"abc"	UTF-8
bool	比較演算の結果	
actor(C)	new C() の結果	クラス C のインスタンス
array	array_empty() 等	要素は同型
record	{a: 1, b: "x"}	構造的型, フィールド型を保持
tuple	(1, "two", 3.0)	位置別型, 長さ固定
future	future expr.m()	後で await で取り出す
unit	print(...) の戻り	

typeof(x) で実行時に型名 (文字列) を取得できる。

3.3 式と文

式

```
expr := リテラル
| 識別子                      // 変数参照、self、sender
| expr op expr                // 二項演算
| new C(arg, ...)             // アクター生成 (式中)
| f(arg, ...)                 // 組込み関数呼び出し
| { l1: e1, l2: e2, ... }     // レコードリテラル
| ( e1, e2, ... )            // タプル (>=2 要素)
| expr . field                // フィールドアクセス
| expr [ n ]                  // タプル / 配列インデックス
| now target.m(args)          // 同期メッセージ送信
| future target.m(args)       // 非同期送信 -> future
| await expr                  // future 値を取り出す
| ( expr )
```

文

```
stmt := var x = expr ;        // ローカル変数宣言
| x = expr ;                  // 代入
| f(arg, ...) ;               // 関数呼び出し (戻り値破棄)
| call f(arg, ...) ;          // 同上 (明示形)
| send target.method(args) ;   // 非同期メッセージ送信
| send! target.method(args) ;  // 型検査をバイパスする送信
| if (expr) stmt [else stmt]
| while expr do stmt
| { stmt; stmt; ... }          // ブロック
| select { case ... timeout ... }
```

if の条件は expr で, ==, !=, <, <=, >, >= を組み合わせる。

3.4 クラスとアクター

```
class ClassName {
  // フィールド宣言 (初期化必須)
  var fieldA = 0;
  var fieldB = 0.0;
  float fieldC = 1.5;    // 型付き宣言 (float のみ専用キーワード)

  method init(args...) { ... }    // インスタンス生成時に自動起動
  method other(args...) { ... }
}
```

- フィールドは **必ず初期化付き**で宣言する
- `init` メソッドが定義されていれば `new` 時に自動で実行される
- 未定義の場合、生成時のメッセージはスキップされ警告ログが出る

トップレベルで実体化:

```
var name = new ClassName(args);
```

`name` がそのままアクター名 (メールボックス参照) になる.

3.5 メッセージ送信

send — 非同期メッセージ送信 (推奨)

```
send target.method(args);
```

- `target` はトップレベルで作ったアクター変数か, `self` / `sender`
- 呼び出しは即座に戻り、メッセージは相手のメールボックスに積まれる
- 型検査が通ったメッセージのみ送る

send! — 型検査をバイパス

リモート相手やインタプリタが完全に型を見切れない動的状況のためのエスケープハッチ. 通常は **send** を使う.

call — 組込み関数の呼び出し

組込み関数 (`print`, `sdl_*`, `wait` など) に対しては `call f(...)` または単に `f(...)`; を使う. これは戻りを待つ通常の関数呼び出しで、他アクターのメソッド呼び出しではない.

now / **future** / **await** (同期・非同期返答付き送信)

```
var v = now calc.add(3, 4) timeout 1000 else 0;    // ブロックして reply 値を取得
var f = future ai.ask("long task"); // すぐ Future 値を返す
var ans = await(f) timeout 1000 else 0;           // 後で値を取り出す
```

ABCL/1 で言うところの `now` 型送信と `future` 型送信に対応する.

3.6 select による選択受信

メールボックスから 特定のメッセージだけを待ち受けたい場合に使う.

```
select {
  case add(a, b) -> {
    reply(a + b);
  }
  case mul(a, b) -> {
    reply(a * b);
  }
  timeout 15000 -> {
    print("15 秒待っても来なかった");
  }
}
```

- 各 case のパターンは メソッド名 (変数名, ...) だけを書ける (ガード式は無し)
- 一致しないメッセージはキューに残る (消費されない)
- timeout N はミリ秒. N 経過すると対応するブロックが実行される
- timeout 節は省略可 (その場合は永久に待つ)

3.7 振る舞いの切り替え — become は無くなった

かつて become でアクター自身を別のクラスに「化ける」ことができたが, この機能は言語から外された. OCaml 版でも Python 版でも構文エラーになる.

外した理由は三つである. ABCL/1 に無かったこと, 型を守る制限を課すと状態フィールドと分岐で書くのとほぼ同じになること, そして 543 本の実プログラムを数えて一つも使われていなかったことである (使っていたのは become 自身のテスト 4 本だけであった).

振る舞いの切り替えは, 状態フィールドと if で書く.

```
class Door {
  var st = 0; // 0=closed 1=opening 2=opened
  method push() : unit !{mut, log} {
    if (st == 0) { st = 1; print("opening"); }
    else { if (st == 1) { st = 2; print("opened"); }
          else { st = 0; print("closed"); } }
  }
}
```

こう書くと, アクター型が実行時に変わらないので外から見た型が嘘にならない. 代価は, 状態が増えると if が深くなることである.

3.8 reply と sender

メソッド本体では暗黙の変数 sender が使える (直近にこのメッセージを送ったアクター).

```
method add(a, b) {
  reply(a + b); // sender に "戻り値" メッセージを送る
}
```

- reply(v) は sender 側に値 v を返す
- 送信側が Web ゲートウェイ (HTTP) 経由なら, HTTP 応答ボディとして返る
- 送信側がアクターなら, 待ちパターン (select) に届く

send sender.X(args) のように sender に対して任意のメソッドを送ることも可能.

3.9 リモート送信

別ホストや別プロセスのアクターには次の構文で送れる。

```
send remote("localhost:8080", "fork2").take(0);
```

- 第 1 引数: ホストとポート (OCaml 側 web_listen で待ち受ける)
- 第 2 引数: 相手プロセス内のアクター名

3.10 レコードと組型 (タプル)

レコード型と組型は拡張層である (Python 版のみ)。核だけで書きたい場合は使わないこと。

OCaml ランタイムでは構造的なレコードとタプルがリテラルとフィールド/インデックスアクセスとともに利用できる。HM 型推論が型を構造的に保持する。

```
// レコード: ラベル付きフィールド
var alice = {name: "alice", age: 30};
print(alice.name);           // "alice"
print(alice.age);            // 30
var p = {pos: {x: 3, y: 4}, label: "origin"};
print(p.pos.x);               // 3 -- チェーンアクセス可

// 組型 (タプル): 位置別型, >=2 要素
var t = (1, "two", 3.5);
print(t[0]);                  // 1
print(t[1]);                  // "two"
print(t[2]);                  // 3.5
var pair = ((10, 20), "label");
print(pair[1]);               // "label"
```

型表現:

- レコード: {name : string; age : int}
- 組型: (int * string * float)

両者とも HM 推論で unify 時にラベル/位置と要素型を照合する。

4 核の七つの規律

型・効果・期限の上に、三実装すべてが備える七つの規律が載っている。どれも注釈を書かなくても効くように作ってある。ここでは要点だけを示す。詳細と例は正典のユーザズガイド (aios/abclcp/docs/aip1_guide_ja.pdf) にある。

規律	書き方	止めるもの
効果	!{ai, net}	実時間側が機外のモデルを待つこと
期限 失敗の型	timeout n else e timeout n (else 無し)	返らない待ち 期限切れの値を正常値として流すこと
返信先の一級性	replyto / answer / 引数型 reply	返信の取りこぼしと二重返信
資源の順序	acquire / release / resource_order	放し忘れ・二重取得・逆順の取得
義務レベル	@n (書かなければ推論)	待ちの循環
セッション型	protocol_define / _start / _end	やり取りの順序違反

4.1 待ちには期限が要る

現行仕様では、待ちに `timeout` を書く。書かないと警告が出る。

```
var v = now calc.add(3, 4) timeout 1000 else 0; // else あり -> int
var r = now calc.add(3, 4) timeout 1000;      // else なし -> result<int>
if (is_ok(r)) { print("ok " ++ value(r, -1)); } else { print("timed out"); }
```

`else` を書かないと `result< $\tau$ >` が返る。これは数値として使えないので、確かめずに計算へ流すことができない。時間切れの値が正常な答えのふりをして下流へ流れる、という壊れ方がここで止まる。

4.2 待ちは必ず「上」へ向かう — 義務レベル

各メソッドに番号を付け、待ちは必ず番号の大きい方へ向かうことを要求する。書かなくてよい。書かなければ、待ちの辺を見て押し上げる不動点で推論される。

```
class Leaf { method f(x) : int @2 !{} { reply(x + 1); } }
class Mid { var l = new Leaf();
            method g(x) : int @1 !{} { reply(now l.f(x) timeout 200 else 0); } }
```

これにより、待ちの宛先がクラスとして分かる範囲ではデッドロック自由が型で保証される。紙の上だけの主張ではなく、Coq/Rocq で `await` を含んだまま機械検証してある (aios/abclcp/coq/AIPLSoundness2.v, 公理なし)。

4.3 資源は対で取り、順序を守る

```
acquire("i2c"); ... ; release("i2c"); // 対になっていないと型エラー
```

さらに、 r を持っているあいだに s を取ったという入れ子があるまま $r < s$ の主張になる。プログラム全体から集めて閉路が無いことを見るので、二つのアクターが同じ資源を逆順に取ると弾かれる。

4.4 やり取りの順序 — セッション型

```
protocol_define("Pipeline", "f.get -> r.scale -> st.put");
var sid = protocol_start("Pipeline");
... ;
protocol_end(sid);
```

実行時のプロトコル宣言を型検査の側でも読む。新しい宣言は要らない。同期の呼び先は展開するので、セッションがアクターをまたいでも追える。

4.5 sender は Python 版だけの機能である

本書の §3.8 で使う `sender` を値として扱う書き方 (`var w = sender;` のように保持する) は拡張層である。核では `sender` は送信の宛先としてしか書けない。

核で「送り主を溜めて後で返す」を書くには、返信先を一級の値として扱う `replyto` を使う。ただし核でもフィールドには持てない (引数としては渡せる)。

5 組込み関数リファレンス

`src/eval_thread.ml` の `prim_table` に登録されている関数群である。すべて `f(args)` または `call f(args);` で呼び出す。

5.1 入出力

関数	説明
<code>print(v)</code>	値を文字列化して標準出力へ. Web UI のログにも残る
<code>typeof(v)</code>	型名 ("int", "float", "string", "actor(C)" ...) を返す

5.2 制御

関数	説明
<code>wait(ms)</code>	現在のアクター (スレッド) を ms ミリ秒スリープ

5.3 数学関数

`sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `sqrt`, `exp`, `log10`, `abs`, `floor`, `ceil`, `round` — いずれも `float` → `float`.

5.4 配列

関数	説明
<code>array_empty()</code>	空配列を生成
<code>array_len(a)</code>	要素数
<code>array_get(a, i)</code>	i 番目の要素 (範囲外は例外)
<code>array_set(a, i, v)</code>	i 番目を v に置換した 新しい配列 を返す
<code>array_push(a, v)</code>	末尾に v を追加した 新しい配列 を返す

配列は永続的データ構造として扱われる (破壊的更新は無い). したがって `a = array_set(a, 0, 99.)`; のように再代入する.

5.5 SDL 描画

関数	説明
<code>sdl_init(w, h)</code>	ウィンドウを開く
<code>sdl_clear()</code>	画面クリア
<code>sdl_present()</code>	フロントバッファ反映
<code>sdl_line(x1,y1,x2,y2)</code>	白線
<code>sdl_line_c(x1,y1,x2,y2,r,g,b)</code>	RGB 指定線
<code>sdl_erase_line(x1,y1,x2,y2)</code>	線を消去 (背景色で再描画)
<code>sdl_poll_key()</code>	キーのスキャンコード (押されてなければ 0)
<code>sdl_mouse_x()</code> / <code>sdl_mouse_y()</code>	マウス座標
<code>sdl_mouse_down()</code>	左ボタン状態 (0/1)

5.6 Web ゲートウェイ

関数	説明
<code>web_listen(port)</code>	内蔵 HTTP サーバを起動
<code>web_expose(path, actorName)</code>	フレンドリーパスを公開
<code>reply(v)</code>	HTTP リクエスト発のメッセージなら応答ボディとして返す

`POST /api/json/send` または `POST /api/x/<path>` で外部からアクターを呼べる.

5.7 AI 呼び出し (ai_call)

```
ai_call("hello")           // 自動選択 (デフォルト Gemini)
ai_call(1, "hello")        // Gemini
ai_call(2, "hello")        // Claude
ai_call(3, "hello")        // ChatGPT
ai_call_with_system(2, "be brief", "hi")
```

第一引数の整数 (省略可) でプロバイダを切り替える:

値	プロバイダ
1 / 省略	Gemini (デフォルト)
2	Anthropic Claude
3	OpenAI ChatGPT

ABCL_AI_PROVIDER=mock を設定すれば全部 mock に流れる (テスト/デモ用).

3 並行モデル経由の AI 呼び出しもそのまま使える:

```
var ans = now ai.ask("hello") timeout 5000 else ""; // 同期 reply 待ち
var fut = future ai.ask("long task"); // future 取得
var done = await(fut) timeout 5000 else ""; // 後で取り出す
send ai.ask("fire and forget"); // past 型, 戻り値不要
```

5.8 デバッグ

関数	説明
actor_dump(a)	アクターの型情報 (クラス名・メソッド一覧) を整形出力

6 サンプルプログラム

6.1 Hello — 最小サンプル

abclc/Hello.aipl

```
class Hello {
  var count = 0.;

  method init(n) {
    count = n;
    print("Hello object initialized with " + n);
  }

  method greet() {
    print("Hello! count = " + count);
  }

  method inc() {
    count = count + 1.;
    print("count incremented to " + count);
  }
}

var h = new Hello(5); // init(5) が自動呼び出し
send h.greet();       // Hello! count = 5
send h.inc();          // count incremented to 6
send h.greet();       // Hello! count = 6
```

学べること: フィールド宣言, `init` の自動呼び出し, `send` による非同期メッセージ, 文字列連結 +.

```
make run-hello
```

6.2 Counter — 自己メッセージとフィールド

abclc/counter.aipl

```
class Counter {
  var count = 0.;
  method inc() {
    count = count + 1.;
    print("count:" + count);
    send self.dec(3.);
  }
  method dec(x) {
    count = count - x;
    print(count);
  }
}

var c1 = new Counter();
var c2 = new Counter();
send c1.inc();
send c2.inc();
```

学べること: `self` を使って自分自身にメッセージを送る, 複数アクターが独立したメールボックスを持つこと.

6.3 PingPong — 二者間メッセージ往復

abclc/PingPong.aipl

```
class Pinger {
  method init() {
    print("Pinger starting");
    send ponger.ping();
  }
  method pong() {
    print("Pinger got pong");
    send sender.ping();
  }
}

class Ponger {
  method ping() {
    print("Ponger got ping");
    send sender.pong();
  }
}

var pinger = new Pinger();
var ponger = new Ponger();
```

学べること: `sender` による「直前の送信元」への返信, 循環メッセージでずっと走り続けるアクター. `init` の中から外部アクターへ `send` できる.

6.4 Door — 状態フィールドによる振る舞いの切り替え

docs/samples/paper/s08_state_machine.aipl

```
class Door {
  var st = 0; // 0=closed 1=opening 2=opened
  method push() : unit !{mut, log} {
    if (st == 0) { st = 1; print("opening"); }
    else { if (st == 1) { st = 2; print("opened"); }
          else { st = 0; print("closed"); } }
  }
  method show() : unit !{log} {
    if (st == 0) { print("closed"); }
    else { if (st == 1) { print("opening"); } else { print("opened"); } }
  }
}
var d = new Door();
send d.show();
send d.push();
send d.push();
```

```
closed
opening
opened
```

学べること: かつて become で書いていた「振る舞いの入れ替え」は, 状態フィールドと分岐で書く. アクター型が実行時に変わらないので, 外から見た型が嘘にならない.

6.5 Records — レコード型

abcllc/Records.aipl (OCaml ランタイムで追加された新機能)

```
var alice = {name: "alice", age: 30};
print("alice.name = " + alice.name);
print("alice.age = " + alice.age);

// ネストレコードとチェーンアクセス
var p = {pos: {x: 3, y: 4}, label: "origin"};
print("p.label = " + p.label);
print("p.pos.x = " + p.pos.x);
print("p.pos.y = " + p.pos.y);
```

学べること: 構造的レコード型 (TRecord) の生成と参照. HM 推論はラベル名とフィールド型を完全に保持し, unify 時にラベル集合と各型を厳密に照合する.

6.6 Tuples — 組型

abcllc/Tuples.aipl

```
var t = (1, "two", 3.5);
print("t[0] = " + t[0]);
print("t[1] = " + t[1]);
print("t[2] = " + t[2]);

var pair = ((10, 20), "label");
print("pair[1] = " + pair[1]);
```

学べること: 位置別型を持つ組型 (TTuple) の作成. インデックスはリテラル整数のみ (静的に位置を解決し, 要素型を取り出す).

6.7 BoundedBuffer — 生産者・消費者問題

abcl/bounded_buffer.aipl (抜粋)

```
class Buffer {
  var cap = 4;
  var s0 = 0; var s1 = 0; var s2 = 0; var s3 = 0;
  var head = 0; var tail = 0; var count = 0;
  var pwaiter = ""; var pitem = 0;
  var cwaiter = "";

  method put(item) {
    if (cwaiter != "") {
      send sender.put_ok();
      send cwaiter.got(item);
      cwaiter = "";
    } else {
      if (count == cap) {
        pwaiter = sender;    // 満杯 → 生産者を待たせる
        pitem = item;
      } else {
        // ... スロットに格納 ...
        send sender.put_ok();
      }
    }
  }
}

method get() {
  // ...
}
```

学べること:

- ロックを **使わず**に同期する手法 (待機キューを自前のフィールドに保持)
- sender を保存しておき、後で send pwaiter.put_ok(); のように「呼び出し側に応答を返す」非同期パターン
- 配列ではなくスロット変数で容量 4 を表現 (言語の最小機能で書ける例)

6.8 Philosophers — 食事する哲学者

abcl/Philosophers5.aipl (5 哲学者). 5 つのアクター間の対称デッドロックを、ハンドオフによって回避する古典問題. SDL 版 (Philosophers5Gui.aipl) ではビジュアライズも可能.

```
make run-philosophers    # コンソール
./run_viz_philosophers.sh # SDL ビジュアル
```

6.9 Rotate4Lines — SDL 描画とタイマー

abcl/Rotate4Lines.aipl

```
class Line {
  var cx = 0.0; var cy = 0.0;
  var angle = 0.0; var len = 50.0;
  var r = 255; var g = 255; var b = 255;
  var x1 = 0.0; var y1 = 0.0; var x2 = 0.0; var y2 = 0.0;
  var drawn = 0;

  method init(startCx, startCy, startAngle, cr, cg, cb) {
```

```

    cx = startCx; cy = startCy; angle = startAngle;
    r = cr; g = cg; b = cb;
}

method rotate() {
  if (drawn == 1) { call sdl_erase_line(x1, y1, x2, y2); }
  angle = angle + 3.0;
  var rad = angle * 3.14159 / 180.0;
  var dx = cos(rad) * len;
  var dy = sin(rad) * len;
  x1 = cx - dx; y1 = cy - dy;
  x2 = cx + dx; y2 = cy + dy;
  call sdl_line_c(x1, y1, x2, y2, r, g, b);
  call sdl_present();
  drawn = 1;
  call wait(32);
  send self.rotate();
}
}

sdl_init(500, 500);
var li1 = new Line(125.0, 125.0, 0.0, 255, 80, 80);
var li2 = new Line(375.0, 125.0, 90.0, 80, 255, 120);
var li3 = new Line(125.0, 375.0, 180.0, 80, 160, 255);
var li4 = new Line(375.0, 375.0, 270.0, 255, 200, 40);
send li1.rotate(); send li2.rotate(); send li3.rotate(); send li4.rotate();

```

学べること: wait(ms) と send self.rotate() の組み合わせによる「自前のタイマーループ」, 4 つのアクターが独立 30FPS で回るマルチスレッド描画.

6.10 WebCalc — HTTP ゲートウェイと select

src/web_calc1.aipl

```

class Calc {
  method init() {
    print("Calc initialized");
    send self.main();
  }

  method add(a, b) {
    reply(999); // すぐ来た add は 999 を返す
  }

  method main() {
    print("waiting...");
    select {
      case add(a, b) -> {
        reply(a + b); // main 待ちの間に来た add だけ正規計算
      }
      timeout 15000 -> {
        print("timeout occurred");
      }
    }
    print("select finished");
  }
}

var calc = new Calc();
web_listen(8080);

```

```
web_expose("/calc", "calc");
print("Open http://localhost:8080/ and send to actor 'calc'");
```

学べること:

- select で「特定のメッセージだけ」を待つ書き方
- timeout 節
- reply で HTTP 応答に値を返す
- web_listen + web_expose による外部 API 化

ブラウザで `http://localhost:8080/` を開き, actor `calc` に `{"method": "add", "args": [3, 4]}` を POST すると 7 が返ってくる.

7 Phase C-E2 型推論 (Python ランタイム)

本章は拡張層である (Python 版のみ)。where による精緻化型を含む。核だけで書きたい場合は使わないこと。

Phase 11 系の gradual 静的型検査に加え, Python 版 AIPL は Phase C 以降で **constraint-based Hindley–Milner 型推論 + Z3 refinement** を備える。注釈の明示が無くてもプログラム全体の型情報を組み立て、後段の Phase E-2 統合 CLI (`--check`) で type-check と inference を同時に走らせられる。

実装ファイル: `src/python-aipl/aipl_inference.py` (約 1255 LOC, Phase E-2 時点).

7.1 CLI: `--infer` と `--check`

フラグ	役割
<code>--type-check</code> <code>--infer</code>	Phase 11 系の signature-based gradual checker (既存) constraint-based HM 推論 + Z3 refinement を走らせ、メソッド毎に推論結果を表示して終了
<code>--check</code>	上記 2 つを section ヘッダ付きで 同時実行 (Phase E-2). 両方の結果を 1 コマンドで得る
<code>--strict</code>	issue があれば exit code 2 (typeck) / 3 (infer) で停止

```
# 型推論のみ
python3 src/python-aipl/aipl_main.py program.aipl --infer

# 統合: type-check + 型推論 (推奨)
python3 src/python-aipl/aipl_main.py program.aipl --check

# 厳密モード: いずれかが issue を出せば exit
python3 src/python-aipl/aipl_main.py program.aipl --check --strict
```

`--infer` / `--check` はプログラムを実行しない (パース後に解析だけ走らせて exit).

7.2 推論される情報

`--infer` の出力例 (`feature_b_crossclass/sample1_simple.aipl` より):

```
=== Adder.add ===
params:
  x : Int
  y : Int
return : Int
```



```

=== Bridge.use_adder ===
  params:
    other : Adder
    a : Int
    b : Int
  return : Int
  locals:
    s : Int

[infer] 2 method(s), 0 unify issue(s), 0 refinement issue(s)

```

注釈ゼロのコードから `Adder.add` の `Int → Int → Int` と `Bridge.use_adder` の `other : Adder` までを **クラス越境で逆推論**している。

7.3 達成された機能 (Phase C → E-2)

機能	Phase	説明
基本 HM 型推論	C	<code>Int → Int</code> の関数や <code>Bool</code> 述語, <code>Rat/Real</code> の自動推論
Cross-class 推論	D-1	<code>now obj.method()</code> 経由で型がクラス間を流れる
where 句 (refinement)	E- α	<code>Int where k >= 0</code> を AIPL 表層で記述, Z3 検査
Actor field 共有	E- β	<code>var n = 0;</code> のクラス field を method 横断で 1 TVar に固定
Record structural	E- γ	<code>{a: Int, b: Str}</code> 構造的型 + width subtyping
Real/Rat refinement	E- γ -R	<code>Real where x > 0.0</code> の Z3 Real 理論判定
typeck × inference 統合	E-2	<code>--check</code> で 1 コマンド, <code>BUILTIN_SIGNATURES</code> 共有

7.4 where 句の使い方

```

class Engine {
  method process(r: Int where r >= 0 and r <= 100) -> Int {
    reply(r * 2);
  }
}

```

AIPL_v2_TypeInference 仕様で進化計算により発見された設計. 充足不可能な refinement (vacuous false) は **declaration-time** に検査される:

```

method bad(k: Int where k >= 5 and k <= 3) -> Int { reply(k); }
// -> refinement issue: vacuously false (Z3 unsat)

```

7.5 詳細ドキュメント

各 Phase の設計記録は [aice-pi-evolution/experiments/2026-05-17_aipl_v2_type_inference/](#): `PHASE_C_REPORT.md` (HM + Z3), `PHASE_D_REPORT.md` (cross-class), `PHASE_E_REPORT.md` (where 句), `PHASE_E_BETA_REPORT.md` (actor field), `PHASE_E_GAMMA_REPORT.md` (record structural), `PHASE_E_GAMMA_R_REPORT.md` (Real/Rat), `PHASE_E_2_REPORT.md` (`--check` 統合), `SAMPLES_SNAPSHOT.md` (7 feature × 3 = 21 sample の実行スナップショット).

8 AIPL v2 Distributed Runtime (aipl_dist)

本章は**拡張層**である (Python 版のみ)。核だけで書きたい場合は使わないこと。

aipl_dist.py は AIPL v2 (2) “Distributed” の進化計算で発見された設計 (I0003 balanced / I0023 hang-resilience / I0036 Erlang OTP の 3 候補) を ランタイム層に **opt-in** で重ね合わせるモジュール (約 591 LOC). AIPL 言語仕様は不変, 既存サンプルは無改変で同じ挙動.

実装ファイル: src/python-aipl/aipl_dist.py (新規モジュール), aipl_interp.py (spawn 時の自動 hook, +27 行), aipl_runtime.py (actor 失敗時の自動 hook, +13 行).

8.1 マスター切替

```
export AIPL_DIST_ENABLE=1 # これが unset / "0" なら全機能 no-op
```

AIPL_DIST_ENABLE=1 を立てないと, aipl_dist の関数群は **すべて即 None / False を返す no-op**. 既存プログラムへの影響ゼロを構造的に保証する.

8.2 8 機能の一覧 (AIPL_DIST_ENABLE=1 前提)

機能	env var	説明
I-1 env_var_routing	AIPL_ROUTE="Name1:tag1,..."	actor 名 → tag のルーティングテーブル
I-2 structured_log	AIPL_DIST_LOG_FILE=/path	ND-JSON 1 行 / event, スレッドセーフ書込
I-3 token_budget_aware	AIPL_DIST_RPM / AIPL_DIST_TPM	60 秒スライディング窓で AI call をレート制限
I-4 checkpoint_and_resume	AIPL_DIST_CHECKPOINT_DIR=/path	actor field を atomic に save, spawn 時 restore
IQ quarantine_and_skip	AIPL_DIST_QUARANTINE_TTL=60	actor 失敗時に自動隔離, 後続 send は skip
IM-1 quorum_replicate	AIPL_DIST_QUORUM_PROVIDERS="..."	並列で N provider に投げ, 最初に応答したものを採用
IM-2 subtree_quarantine	AIPL_DIST_SUBTREE_QUARANTINE=1	actor 失敗時にその子孫も一括隔離 (Erlang OTP 風)
integration	(上記の組合せ)	spawn ツリー追跡 + 観測性 + レジリエンス

8.3 クイックスタート

AIPL_DIST_ENABLE=1 一発で観測性 (I-2) と spawn ログだけが ON になる:

```
AIPL_AI_PROVIDER=mock AIPL_DIST_ENABLE=1 \
AIPL_DIST_LOG_FILE=/tmp/aipl.ndjson \
python3 src/python-aipl/aipl_main.py program.aipl

cat /tmp/aipl.ndjson
# {"ts": ..., "event": "actor_spawn", "actor": "counter", "cls": "Counter"}
# {"ts": ..., "event": "actor_routed", "actor": "counter", ...}
```

8.4 PsiLang v3 silent-hang シナリオへの対策一式

PsiLang v3 trial #1 で OpenAI silent hang により 35/38 個体喪失した問題に対し, 3 MVP を同時有効化すると **call-level / actor-level / subtree-level の 3 重防御**:

```
export AIPL_DIST_ENABLE=1
export AIPL_DIST_QUORUM_PROVIDERS="openai,anthropic,gemini" # IM-1
export AIPL_DIST_QUARANTINE_TTL=60 # IQ
export AIPL_DIST_SUBTREE_QUARANTINE=1 # IM-2
```

```
export AIPL_DIST_CHECKPOINT_DIR=/tmp/aipl_ck          # I-4
export AIPL_DIST_LOG_FILE=/tmp/aipl.ndjson           # I-2
```

- **call-level**: OpenAI が hang しても anthropic/gemini が応答 → caller 継続
- **actor-level**: それでも actor が落ちたら自動 quarantine (60s)
- **subtree-level**: 子孫 actor も一括隔離 (Erlang OTP blast-radius containment)
- **persistent state**: actor 再 spawn 時にチェックポイント復元
- **observability**: 全イベントが NDJSON で記録

8.5 サンプル (8 機能 × 3 = 24)

aiice-pi-evolution/experiments/2026-05-17_aipl_v2_type_inference/IMPL_I0003_MVP/samples/
配下に feature 別サブディレクトリで整理:

```
samples/
i1_env_var_routing/      sample1_basic.aipl, sample2_no_match.py, sample3_log_correlation.py
i2_structured_log/      sample1_basic.py, sample2_no_file.py, sample3_multi_thread.py
i3_token_budget/        sample1_basic.py, sample2_blocking.py, sample3_aipl_integration.
    aipl
i4_checkpoint/          sample1_basic.py, sample2_atomic.py, sample3_list_states.py
iq_quarantine/          sample1_basic.aipl, sample2_ttl_expiry.py, sample3_manual_clear.py
im1_quorum/             sample1_first_wins.py, sample2_tolerates_one_failure.py,
    sample3_all_fail.py
im2_subtree_quarantine/ sample1_basic.aipl, sample2_grandchildren.aipl,
    sample3_unrelated_unaffected.aipl
integration/            sample1_basic.aipl, sample2_combined_logging.aipl,
    sample3_full_resilience.aipl
```

ユニットテストは IMPL_I0003_MVP/tests/test_aipl_dist.py (27 個).

8.6 公開 API (aipl_dist モジュール)

```
import aipl_dist

# Master toggle
aipl_dist.is_enabled()          # -> bool

# I-1 routing
aipl_dist.route_for(name)       # -> Optional[str]
aipl_dist.parse_route_table(raw) # -> dict[str, str]

# I-2 log
aipl_dist.log_event(event, **fields) # -> bool

# I-3 budget
aipl_dist.token_budget_gate()    # -> Optional[TokenBudgetGate]
aipl_dist.call_ai_with_budget(prompt, call_ai_fn=None, **kw)

# I-4 checkpoint
aipl_dist.save_actor_state(name, state) # -> bool
aipl_dist.restore_actor_state(name)     # -> Optional[dict]
aipl_dist.list_actor_states()           # -> dict[str, str]

# IQ quarantine
aipl_dist.quarantine_actor(name, ttl=None) # -> bool
aipl_dist.is_quarantined(name)            # -> bool
```

```

aipl_dist.clear_quarantine(name)          # -> bool
aipl_dist.quarantine_status()             # -> dict[str, float]

# IM Erlang OTP
aipl_dist.register_spawn(child, parent)
aipl_dist.descendants_of(actor)             # -> list[str]
aipl_dist.quarantine_subtree(actor, ttl=None)
aipl_dist.call_ai_quorum(prompt, providers=None, **kw)

```

すべての関数は AIPL_DIST_ENABLE != "1" 時に None / False / 空 dict を返す no-op.

8.7 進化計算による設計探索の出自

aipl_dist.py の設計は手書きではなく, AIPL_v2_Distributed.aice (11 軸 GA 仕様) を MAP-Elites で 8 seed × 30 gen 探索した結果から派生:

- 仕様: AIPL_v2_Distributed.aice / .ga.json / .schema.json
- Run 1 / Run 2 結果: distributed_run_outputs/ (38 個体 / 25 cells / 24 min × 2)
- 設計レポート: IMPL_DESIGN.md, IMPL_RUN_REPORT.md, IMPL_INTEGRATION_REPORT.md, IMPL_I0023_REPORT.md, IMPL_I0036_REPORT.md

9 Next-gen 機能 (CE-10~13 + DR-10~13, 2026-05-18)

本章は拡張層である (Python 版のみ)。核だけで書きたい場合は使わないこと。

Phase C-E2 (CE-1~9) と AIPL v2 Distributed (DR-1~9) の上に積み増した **8 個の next-gen 機能**. 進化計算 (round 4 + 5) で 4 つの type-system 軸 (CE-10~13) と 4 つの分散軸 (DR-10~13) が選定され, Py-I → OCaml → JS-O → JS-B / JS-N → C の 6 ランタイム全てに ポート完了 (docs/AIPL_Runtime_Feature_Matrix.pdf 18 ページ参照). 既存 26 機能と全 71 サンプルは無修正で動作.

9.1 8 機能と既定値

機能	既定値 / トリガ	役割
CE-10 effect	自動 (常時 on, opt-out 不要)	各 method の副作用集合を {ai, fs, net, mut} で推論し --check 出力に表示. 外部効果を持つメソッドを誤って pure 扱いするのを防ぐ.
CE-11 capability	AIPL_CAP_GRANT="fs,ai,... strict AIPL_CAP_STRICT=1	primitive 呼出し前に check_capability("fs") を挟むと, cap が grant されていなければ cap_violation ログ + (strict 時) 例外.
CE-12 refinement unify	既定 lazy, AIPL_REFINE_UNIFY=1 で eager	int where x>0 などの述語を unify 時に Z3 で subset 判定. JS-N は AIPL_REFINE_Z3=1 で z3 spawn, JS-B は gradual (z3-wasm 未バンドル).
CE-13 record sub-typing	自動	{name, age, email} を p: {name: string} に渡せるよう intersection-based に緩和 (depth 互換は維持).
DR-10 CRDT state	自動 (AIPL_DIST_ENABLE=1 下で)	G-Counter / OR-Set / LWW-Register の 3 種 + crdt_replicate. multi-replica state を競合なく合流.
DR-11 saga	自動 (構文 saga { step {...} compensate {...} ... })	step 列の途中で例外が起きると, 既完了 step の compensate を LIFO で発火し再 raise. saga_* 7 イベントを NDJSON で記録.
DR-12 region failover	AIPL_REGION=us-east-1, AIPL_REGION_FAILOVER="us-east-1, AIPL_ROUTE_REGION_<R>="ac-to-east-1,..."	failover_region(actor) がチェーンを順次試行し, 見つかった region を返す.
DR-13 auto-scaling pool	自動 (pool_create(cls, min, max, target_qlen))	hysteresis controller で qlen 過多なら spawn, 空きすぎなら retire. pool_pick はラウンドロビン.

9.2 CE-10 effect 推論の出力例

```
$ aipltc --check effects_demo.aipl
=== inferred method effects ===
[class_method_effects]
class AIReporter
  summarise effects: {ai, mut}
class FileLog
  write effects: {fs, mut}
  peek effects: {fs}
class Pure
  add effects: {}
```

OCaml ランタイムでは repl_thread.exe --check で同じ出力. Py-I は python3 src/python-aipl/aipl_main. --check FILE, JS-N は POST /api/typecheck の JSON で methods["m"] = "(...) -> _ ![ai, fs, mut]" として返る.

9.3 CE-11 capability の使い方

```
export AIPL_CAP_GRANT="fs,ai"
export AIPL_CAP_STRICT=1
```

```
aipl2c run.aipl && cc run.c -o run && ./run
```

run.aipl 側では:

```
class Worker {
  method run() {
    check_capability("fs");      // ok: grant に入っている
    var c = read_file("/etc/hosts");
    check_capability("net");     // FAIL: strict mode で throw
    // ...
  }
}
```

strict mode を切ると check_capability は cap_violation ログを出すだけで進む。

9.4 DR-11 saga 構文

step { body } compensate { rollback } のペアを並べる。

```
class Flow {
  method run(b) {
    saga {
      step      { send b.reserve(2); }
      compensate { send b.release(2); }
      step      { send b.charge(100); }
      compensate { send b.refund(100); }
    }
  }
}
```

2 step 目で例外が起きると compensate 1 が発火して reserve が undo される。NDJSON ログ (AIPL_DIST_LOG_FILE=/path 必須) に 7 種類のイベント (saga_started / saga_step_complete / saga_step_failed / saga_compensated / saga_compensate_failed / saga_finished / saga_aborted) を吐く。

9.5 DR-10 CRDT の使い方

```
class Counter {
  var c = 0;
  method init() { c = crdt_gcounter_new(); }
  method inc(n) { crdt_gcounter_inc(c, n); }
  method val() { reply(crdt_gcounter_value(c)); }
  method sync(other) {
    var merged = crdt_gcounter_merge(c, other);
    c = merged;
    crdt_replicate(c);
  }
}
```

OR-Set (crdt_orset_add / remove / contains / values / merge) と LWW-Register (crdt_lww_new / write / value / merge) も同様の API. crdt_replicate は構造化ログに crdt_replicate イベントを残すフック (実 transport は別途).

9.6 DR-12 multi-region failover

```
export AIPL_REGION=us-east-1
export AIPL_REGION_FAILOVER="us-east-1,eu-west-1,ap-northeast-1"
export AIPL_ROUTE_REGION_us-east-1="Greeter:fast"
export AIPL_ROUTE_REGION_eu-west-1="Greeter:slow"
```

プログラム側で `failover_region("Greeter")` を呼ぶと、チェーンを順に試行して route が見つかった region を返す. 全 region で route 未定義なら `region_failover_failed` をログに残して空文字を返す.

9.7 DR-13 auto-scaling actor pool

```
class Master {
  method init() {
    var pool = pool_create("Worker", 2, 10, 4); // min, max, target_qlen
    // ...
    var w = pool_pick(pool);
    send w.task("payload");
  }
}
```

`qlen` が `target_qlen` を上回ると hysteresis controller が新 worker を spawn, 下回り続けると `retire.pool_size` で現在の worker 数を取得. `pool_destroy` で明示終了 (子 actor は通常通り終了).

9.8 6 ランタイムでの動作

機能ごとの 状況は `docs/AIPL_Runtime_Feature_Matrix.pdf` (18 ページ) が随時更新される正本.2026-05-18 時点で Py-I / OCaml / JS-O / JS-B / JS-N / C 全てに 26 個の next-gen 機能が揃っている (Py-A は意図的に最小実装枠で 17 機能が).

9.9 進化計算による設計の経緯

8 機能は手書きではなく `AIPL_PyI_NextGen.aice / AIPL_OCaml_NextGenPort.aice / AIPL_C_NextGenPort.aice / AIPL_JSBJSN_NextGenPort.aice` の 4 つの仕様を MAP-Elites で round 2~5 として探索した結果:

- Round 2 (Py-I $4 \times 4 = 16$ cell): CE-10..13 + DR-10..13 を選定
- Round 3 (OCaml port): Py-I 8 機能をそのまま OCaml に直訳
- Round 4 (C codegen): `abcl_nextgen_runtime.{c,h}` 880 LOC
- Round 5 (JS-B + JS-N): 共有 `src/browser-abcl/src/` 経由

4 .aice 仕様は `aice-pi-evolution/experiments/2026-05-18_*/`.

10 付録: トラブルシューティング

症状	対処
Unknown function: foo	組込みの綴り誤り. <code>prim_table (src/eval_thread.ml)</code> の登録名と照合
Actor X not found	<code>var X = new ...</code> ; より前に <code>send X....</code> していないか確認
[Actor] X.init arity mismatch	<code>new C(args)</code> の引数数と <code>init</code> の仮引数数が違う
<code>select</code> がいつまでも返らない	パターンが受信メッセージと一致していない/ <code>timeout</code> 未指定
SDL ウィンドウが反応しない	<code>sdl_present()</code> を毎フレーム呼ぶ/ <code>wait</code> で <code>yield</code> する
HTTP リクエストでハング	<code>reply</code> を呼んでいない or <code>select</code> が適切な case を持たない
no overload of foo matches (...)	HM 推論がオーバーロード候補と整合しなかった. 実引数の型を <code>typeof</code> で確認
no field f in record {...}	レコードにそのラベルが無い. ラベル名の綴りを確認
tuple index N out of bounds	組型のサイズと [N] の整合性を確認
[infer] N unify issue(s)	--infer 出力. 位置 (at assign to s 等) を見て注釈を調整
refinement is vacuously false	where 句が Z3 で unsat. 値域を見直す
[actor X.Y] error: ...	実行時 actor 内エラー.AIPL_DIST_ENABLE=1 AIPL_DIST_QUARANTINE_TTL=60 で自動隔離
AIPL ランタイムが silent hang	AIPL_DIST_QUORUM_PROVIDERS="openai,anthropic" で並列フェイルオーバ
OpenAI tier-1 rate limit	AIPL_DIST_RPM=400 AIPL_DIST_TPM=180000 で I-3 ゲートを有効化

設計指針のまとめ: AIPL では「同期したいなら sender に `reply`, 待ちたいなら `select`, 状態を切り替えたいなら状態フィールドと分岐」と覚えておくと, 大半の並行パターンを言語機能だけで表現できる. 型は HM 推論で自動的に決まるため, 通常は注釈を書かずに走らせて構わない. 明示したい場合のみ Python (annotated) ランタイムを使うとよい.